ORACLE NetSuite

Plug-ins Guide

2025.2

November 12, 2025

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at https://www.oracle.com/corporate/accessibility.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info or visit http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and “as available”, for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer’s use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle’s other customers, Oracle may slow down or throttle Customer’s excess use until such time that Customer’s use stays within reasonable limits. If Customer’s particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer’s actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Table of Contents

Custom Plug-in Overview

A custom plug-in is customizable functionality that is defined by an interface. When you define an interface, third-party solution providers can develop custom plug-in implementations and bundle them in a SuiteApp. After the SuiteApp is installed in an account, a solution implementer can define one or more alternate implementations. These implementations let the solution implementer customize the plug-in's logic to suit specific business needs.

Important: The object-oriented interface is central to the plug-in model. To be more exact, a plug-in **is** an interface. Plug-ins aren't APIs and don't expose a class's functions or objects. However, a Plug-in lets a third party override the logic in its default implementation.

A custom plug-in interface defines the function names, their parameters, and return types. You can call Interfaces in any third-party server SuiteScript script, except for Mass Update scripts.

Note: Solution providers can only call functions from a custom plug-in interface within the custom plug-in implementation.

The solution provider defines a custom plug-in's default logic in a default implementation. If applicable, the solution provider may define one or more alternate implementations.

Note: Alternate implementations must be associated with a plug-in implementation type, which is a standard record type in NetSuite. When a solution provider releases a custom plug-in with alternate implementations, solution implementers cannot edit these alternate implementations.

When the custom plug-in's implementations are installed or defined in the user's account, the NetSuite administrator activates the available implementations and chooses which one use. Whether a custom plug-in can use one or multiple implementations at a time depends on how the custom plug-in is set up. When an implementation is active, function calls in the custom plug-in script run that implementation's logic.

Sample Use Case

Consider a SuiteApp that includes logic for calculating asset depreciation. The solution provider turns this functionality into a custom plug-in. The solution implementer then overrides the default functionality with one or more alternate implementations based on the user's specific accounting principles and business needs.

In another scenario, the solution provider releases the custom plug-in with multiple implementations, by-passing the solution implementer role. The NetSuite administrator chooses (activates) which one to run based on the user's needs.

Custom Plug-in Types

A custom plug-in type is a record type in NetSuite that holds a custom plug-in's implementations and any supporting library files. The name of a custom plug-in type helps you distinguish its implementations from other custom plug-in implementations installed on an account.

Solution providers, solution implementers, and administrators use custom plug-in type in the following ways. For more information, see Custom Plug-in Creation.

Solution Providers

- After a solution provider defines an interface's default implementation, they must create a custom plug-in type record for it.
- After a solution provider defines an alternate implementation, they must specify which custom plug-in type it belongs to. They do this by creating a new plug-in implementation record. The new plug-in implementation record is a child record of the custom plug-in type record created for the default implementation.

Solution Implementers

- After a solution implementer defines an alternate implementation, they must specify which custom plug-in type it belongs to. They do this by creating a new plug-in implementation record. The new record is a child record to the custom plug-in type record created for the default implementation.

Administrators

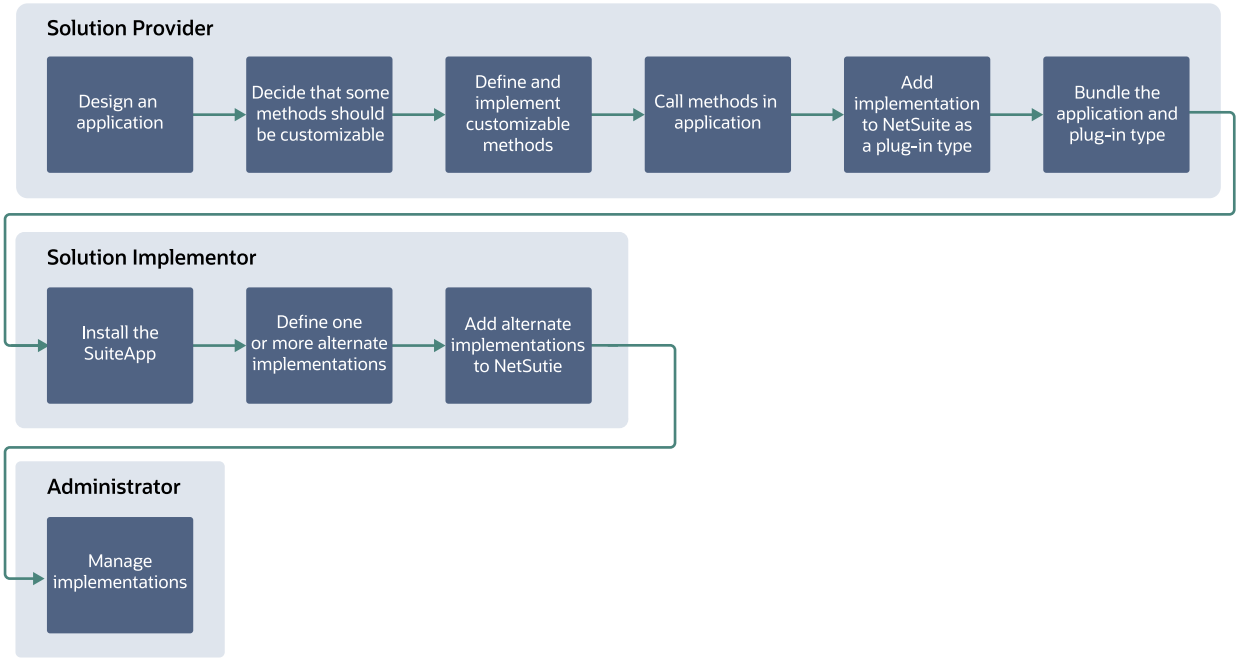
- Administrators use the Manage Plug-In Implementations page to manage all plug-ins installed on an account. Administrators use the custom plug-in type name to distinguish a plug-in's implementations from the those of other custom plug-ins installed on an account. For more information, see the help topic Managing Plug-ins.

Custom Plug-in Creation

Important: To create your own custom plug-ins and implementation scripts, you need some JavaScript experience and an understanding of SuiteScript. You should write new custom plug-in scripts in SuiteScript 2.1. To get started with SuiteScript, see the help topic SuiteScript 2.1.

The following diagram illustrates the basic process for developing, implementing, and managing custom plug-ins. Note that your process may vary from the following diagram. See Suggested Help Topics by Role for a list of help topics organized by role.

Note: This diagram is organized by role, not job title. Roles do not always match titles. For example, one person or group may handle both the solution implementer and administrator roles.



Suggested Help Topics by Role

Solution Provider	Solution Implementor	Administrator
Custom Plug-in Development ■ Creating a Custom Plug-in Interface ■ Creating the Default Implementation for a Custom Plug-in ■ Adding the Default Implementation to NetSuite ■ If you are creating alternative implementations: □ Creating a Custom Plug-in Alternate Implementation	Custom Plug-in Implementation ■ Creating a Custom Plug-in Alternate Implementation ■ Adding an Alternate Implementation to NetSuite	Managing Custom Plug-in Implementations Viewing Plug-in Implementation System Notes

Solution Provider	Solution Implementor	Administrator
◦ Adding an Alternate Implementation to NetSuite ■ Instantiating a Custom Plug-in Script in SuiteScript 2.x ■ Adding a Script that Instantiates a Custom Plug-in to NetSuite ■ Bundling a Custom Plug-in		

Custom Plug-in Development

Important: To create your own custom plug-ins and implementation scripts, you need some JavaScript experience and an understanding of SuiteScript. You should write new custom plug-in scripts in SuiteScript 2.1. To get started with SuiteScript, see the help topic SuiteScript 2.1.

Important: You cannot use the SuiteScript Debugger to ad hoc debug a script that uses the N/plugin Module. You must use deployed debugging instead. To use deployed debugging, you must complete the steps described in Adding a Script that Instantiates a Custom Plug-in to NetSuite. For additional information about ad hoc and deployed debugging, see the help topic SuiteScript Debugger.

As a solution provider, the following sections must be reviewed in the following order.

- Creating a Custom Plug-in Interface
- Creating the Default Implementation for a Custom Plug-in
- Adding the Default Implementation to NetSuite
- If you are creating alternative implementations:
  - Creating a Custom Plug-in Alternate Implementation
  - Adding an Alternate Implementation to NetSuite
- Instantiating a Custom Plug-in Script in SuiteScript 2.x
- Adding a Script that Instantiates a Custom Plug-in to NetSuite
- Bundling a Custom Plug-in

Creating a Custom Plug-in Interface

The interface is central to the custom plug-in model. A custom plug-in's interface defines the functions that run within the custom plug-in script. A custom plug-in script can be any server script type except a Mass Update script. Note that client scripts can't be used as custom plug-in scripts.

Important: Functions defined in a custom plug-in's interface are only called within the custom plug-in script's code.

The functions in an interface are not fully defined. Each function includes a signature (the function name and parameters) and a return type, but no body.

An implementation fully defines each function in the interface and the logic they run. You must define a default implementation of the interface. If needed, you can also define one or more alternate implementations of the interface.

Important: Each implementation must keep the signature and return type defined in the interface.

Creating the Default Implementation for a Custom Plug-in

Important: To create your own custom plug-ins and implementation scripts, you need some JavaScript experience and an understanding of SuiteScript. You should write new custom plug-in scripts in SuiteScript 2.1. To get started with SuiteScript, see the help topic SuiteScript 2.1.

An implementation fully defines an interface's functions and contains the logic they run. When you're developing a plug-in, you must define a default implementation to set up the custom plug-in script's default logic. The default implementation is written as an independent JavaScript file.

Note: Each function in the interface must be fully defined in the default implementation.

Tip: Programming Note: Log messages written by using N/log Module functions (log.debug, log.error, log.audit, log.emergency) in a plug-in implementation are tracked on the script record that calls the plug-in. The log level is determined by that script's deployment. You can see log messages by viewing the execution log of the script that called the plug-in. For more information about script deployment, see the help topic SuiteScript 2.x Entry Point Script Creation and Deployment.

SuiteScript 2.x Custom Plug-in Default Implementation Example

The following is a SuiteScript 2.x example of a **default implementation**:

```
/**
 * @NApiVersion 2.x
 * @NScriptType plugintypeimpl
 */
define(function() {
    return {
        doTheMagic: function(inputObj) {
            var operand1 = parseFloat(inputObj.operand1);
            var operand2 = parseFloat(inputObj.operand2);
            if (!isNaN(operand1) && !isNaN(operand2)) {
                return operand1 + operand2;
            }
        },
        otherMethod: function() {
            return 'sample';
        }
    }
});
```

This sample default implementation fully defines the `doTheMagic` function. When you call it in the custom plug-in script, this function accepts two numbers, `operand1` and `operand2`, and adds them together to return a result.

Adding the Default Implementation to NetSuite

To add a default implementation to NetSuite, you create a Custom Plug-in Type. A Custom Plug-in Type is a record type within NetSuite that holds a custom plug-in's default implementation and any supporting library files. Attach the default implementation's JavaScript file to the Custom Plug-in Type record to save it. See Creating the Default Implementation for a Custom Plug-in before performing the steps below.

Important: You must have Server SuiteScript enabled and SuiteScript permission to create a custom plug-in type record.

Note: Alternate implementations are also stored as a record type in NetSuite, called the New Plug-in Implementation record, which is a child record of the Custom Plug-in Type record.

To create a custom plug-in type:

1. Go to Customization > Plug-ins > Custom Plug-in Types > New.
2. Select the script file that contains your default implementation, and then click **Create Custom Plug-in Type**.
3. On the custom plug-In type record, enter the following:
   - **Name**: Enter a user-friendly name for the custom plug-in type. Solution implementers and administrators see this name when creating alternate implementations or enabling/disabling implementations.
   - **ID**: Enter an internal ID for the custom plug-in type. If you don't enter an ID, NetSuite creates one for you after you click **Save**.
   - **Class Name (SuiteScript 1.0 scripts only)**: Enter a class name for the custom plug-in using Pascal case (PascalCase, for example, DemoPlugInType). As the solution developer, you'll use this class name to instantiate the implementation you want to use in your custom plug-in script. Make sure the class name is descriptive. For more information, see Instantiating a Custom Plug-in Implementation.

     Note: The term "class name" is misleading. When you create a new instance of an implementation in your custom plug-in script, you are instantiating a delegate, not an object.
   - **Deployment Model**: Specify how many custom plug-in type implementations an administrator can activate at one time.

     Important: The Deployment Model you choose affects how you write your custom plug-in script. For more information, see Instantiating a Custom Plug-in Implementation.

     The Deployment Model field provides the following options:
     - **Allow Multiple**: You can activate multiple implementations of the interface in an account at the same time.
     - **Allow Single**: You can activate only one implementation of the interface in an account at any time.

     Note: The Deployment Model field doesn't limit how many implementations a custom plug-in can have. If this field is set to **Allow Single**, you can still have as many alternate implementations as you want, but only one implementation can be activate at any time.
   - **Status**: Set to either **Testing** or **Release**. Be sure to set the status to **Released** before you bundle your custom plug-in. For more information, see Bundling a Custom Plug-in.
   - **Log Level**: Set to the logging level you want for the custom plug-in script.
   - **Description**: If you choose, enter a short description of what the custom plug-in does.
   - **Owner**: This field defaults to the name of the user who's logged in.
   - **Inactive**: Set whether you want the custom plug-in type to be active or inactive.

- On the **Methods** subtab, add the names of the functions defined from the interface. Enter the function name only without parentheses or parameters.
- On the **Scripts** subtab, add the following:
  - **Default Implementation (SuiteScript 1.0 scripts only)**: Browse to the default implementation's JavaScript file. The lists only shows JavaScript files uploaded to the SuiteScript folder in the NetSuite File Cabinet, but you can also attach your .js file from a local directory or by URL.
  - **Documentation**: Add documentation for your default implementation here. You should write an interface definition that describes the functions in your interface. Solution implementers need this information to create alternate implementations.
  - **Libraries (SuiteScript 1.0 scripts only)**: Add library files that support your alternate implementation here.
- On the **Unhandled Errors** subtab: Specify who is notified if script errors occur. Note that you'll only get notifications for errors in your own (the solution provider's) account. You won't be notified of errors that occur in the accounts of those who install your custom plug-in.

4. Click **Save**.

Note: You can use SuiteCloud Development Framework (SDF) to manage default custom plug-in implementations as part of file-based customization projects. For information about SDF, see .

You can use Copy to Account to copy an one default custom plug-in implementation to another of your accounts. When you edit a default custom plug-in implementation, you'll see a clickable **Copy to Account** option in the upper right corner. For information about Copy to Account, see the help topic Copy to Account.

Instantiating a Custom Plug-in Script in SuiteScript 2.x

Important: To create your own custom plug-ins and implementation scripts, you need some JavaScript experience and an understanding of SuiteScript. You should write new custom plug-in scripts in SuiteScript 2.1. To get started with SuiteScript, see the help topic SuiteScript 2.1.

To instantiate a custom plug-in script implementation in your code, use the `plugin.loadImplementation` function. For more information, see the help topic plugin.loadImplementation(options).

There are two ways to invoke a specific implementation:

- Using SuiteScript, pass the script implementation ID to the `implementation` parameter of the `plugin.loadImplementation` function.
- Using the **Manage Plug-ins** page, instantiate the implementation that is currently active in the customer's account.

Important: To use the Script Debugger on a script that instantiates a specific implementation, you must add the script that instantiates a Custom Plug-in to NetSuite. For more information, see Adding a Script that Instantiates a Custom Plug-in to NetSuite.

Instantiating a Specific Implementation using SuiteScript

Instantiate a specific implementation by passing the script implementation ID to the `implementation` parameter of the `plugin.loadImplementation` function. The ID for the default implementation is `default`. Use the `plugin.findImplementations` function to get a list of all the active script implementation IDs.

The following example shows a Suitelet script that performs these tasks:

- Use the plugin.findImplementations function to count the total number of active implementations and get a list of all the implementation IDs for a custom plug-in script type ID named customscript_magic_plugin.
- Traverse each implementation, instantiating them with the plugin.loadImplementation function and running doTheMagic, a function defined in each implementation.

```
/**
 * @NApiVersion 2.x
 * @NScriptType suitelet
 */
define(['N/plugin'], function(plugin) {
    return {
        onRequest: function(options) {
            var impls = plugin.findImplementations({
                type: 'customscript_magic_plugin'
            });
            for (i = 0; i < impls.length; i++) {
                var pl = plugin.loadImplementation({
                    type: 'customscript_magic_plugin',
                    implementation: impls[i]
                });
                options.response.write('impl = ' + impls[i] + ', result = ' + pl.doTheMagic({
                    operand1: 10,
                    operand2: 20
                }) + '\n');
            }
        }
    }
});
```

For more information about the plugin.findImplementations function, see the help topic plugin.findImplementations(options).

Instantiating a Specific Implementation using the Manage Plug-ins Page

To instantiate a specific implementation using the UI, leave out the `implementation` parameter in your plugin.loadImplementation call. If you don't specify this parameter, NetSuite runs the active plug-in implementation that is specified by the **Manage Plug-ins** page.

The following example shows a Suitelet script that runs doTheMagic, a custom plug-in function that is defined in every implementation of the customscript_magic_plugin type. The `implementation` parameter isn't specified.

```
/**
 * @NApiVersion 2.x
 * @NScriptType suitelet
 */
define(['N/plugin'], function(plugin) {
    return {
        onRequest: function(options) {
            var pl = plugin.loadImplementation({
                type: 'customscript_magic_plugin'
            });
            options.response.write('impl not specified, result = ' + pl.doTheMagic({
                operand1: 10,
                operand2: 20
            }) + '\n');
        }
    }
});
```

To specify which script implementation to run:

1. Go to Customization > Plug-ins > Manage Plug-ins.
2. Find your custom plug-in, and then select the active plug-in you want from the list of implementations.
3. Click **Save**.

Instantiating a Custom Plug-in Script in SuiteScript 1.0

Important: To create your own custom plug-ins and implementation scripts, you need some JavaScript experience and an understanding of SuiteScript. You should write new custom plug-in scripts in SuiteScript 2.0 or SuiteScript 2.1. To get started with SuiteScript, see the help topic SuiteScript 2.1.

Instantiating a Custom Plug-in Implementation

When you create a Custom Plug-in Type record in Adding the Default Implementation to NetSuite, you enter a **Class Name** for the custom plug-in. Before you can call an implementation's functions, you must use the class name to create a new instance of the implementation you want to use.

Note: The term "class name" is a misleading. When you create a new instance of an implementation, you're instantiating a delegate, not an object.

Instantiate the implementation using the `new` keyword and the **Class Name** from the custom plug-in type record. The constructor has one optional parameter, `implementationId`, which determines which implementation you instantiate. See Instantiating a Specific Implementation and Instantiating an Implementation Based on the Deployment Model for the available arguments.

```
var a = new <Class Name>(implementationId);
```

Important: You can instantiate active implementations only after the plug-in is installed by a customer and the plug-in script is executing.

There are two techniques you can use to instantiate an implementation:

- You can directly specify the implementation your code is instantiating.
- You can instantiate based on the **Deployment Model** setting on the Custom Plug-in Type record. In other words, you can instantiate the implementations that are currently active in the customer's account.

Instantiating a Specific Implementation

Pass one of the following arguments for `implementationId` to instantiate a specific implementation.

- `default` – Pass this string argument to create an instance of the default implementation. Be sure to enclose the argument in single quotes. See Calling a Default Implementation's Functions for an example.

- The ID entered on the New Plug-in Implementation record – Pass this string argument to create an instance of a specific alternate implementation. Be sure to enclose the argument in single quotes. See Calling an Alternate Implementation's Functions for an example.

Instantiating an Implementation Based on the Deployment Model

Pass one of the following arguments for `implementationId` to instantiate the implementations that are currently active in the customer's account. The argument you use depends on the value set for the **Deployment Model** field. See Adding the Default Implementation to NetSuite for additional information.

- No argument – If you don't pass an argument, the active implementation is instantiated. Use this option if the **Deployment Model** field is set to **Allow Single**.
- An array, returned by `getImplementations`, that contains all active implementations – Use this option if the **Deployment Model** field is set to **Allow Multiple**. You must use the `getImplementations` method to instantiate the implementations. For more information and an example, see Discovering Active Implementations.

Calling a Default Implementation's Functions

```
function useCustomPlugInType()
{
    var a = new DemoPlugInType('default');
    a.doTheMagic({
        operand1: 10,
        operand2: 20
    });
    a.otherMethod();
}
```

Calling an Alternate Implementation's Functions

```
function useCustomPlugInType()
{
    var b = new DemoPlugInType('customscriptimplementation1');
    b.doTheMagic();
    b.otherMethod();
}
```

Calling Multiple Implementations' Functions

In this example, you, the solution provider, create a default implementation and an alternate implementation for the plug-in. The alternate implementation is called by passing the ID from the New Plug-in Implementation record as an argument.

Make sure that the Deployment Model on the custom plug-in type record is set to Allow Multiple.

```
function useCustomPlugInType()
{
    var a = new DemoPlugInType('default');
    a.doTheMagic({
        operand1: 10,
        operand2: 20
    });
    a.otherMethod();

    var b = new DemoPlugInType('customscriptimplementation1');
    b.doTheMagic();
    b.otherMethod();
}
```

Discovering Active Implementations

After your custom plug-in is installed on a customer's account, the custom plug-in script doesn't directly know about the active implementations. If you set the **Deployment Model** field on the Custom Plug-in Type record to **Allow Single**, this is not an issue. See Instantiating an Implementation Based on the Deployment Model for additional information.

If the **Deployment Model** field on the Custom Plug-in Type record is set to **Allow Multiple** and you don't specify which implementation to instantiate, you'll need to use the `getImplementations` method to instantiate the implementations. The `getImplementations` method returns an array of active implementations.

```
function discoverImplementations()
{
    var x = DemoPlugInType.getImplementations();
    for (var i = 0; i < x.length; i++)
    {
        var t = new DemoPlugInType(x[i]);
        t.doTheMagic();
        t.otherMethod();
    }
}
```

Adding a Script that Instantiates a Custom Plug-in to NetSuite

Create a script record for each script that instantiates your custom plug-in script. For more information, see the help topic Creating a Script Record.

On the script record, list the Custom Plug-in Type that represents your implementations.

To reference a custom plug-in type on a script record:

1. Click the **Scripts** subtab. Then click the **Custom Plug-In Types** subtab.
2. In the **Custom Plug-In Type** field, select the custom plug-in type that represents your plug-in's implementations.

Bundling a Custom Plug-in

Custom plug-in implementations are bundled as part of a SuiteApp. For installation instructions, see the SuiteApp documentation. For more information about creating SuiteApps, see the help topic SuiteBundler Overview.

Note: SuiteBundler is still supported, but it won't be updated with any new features.

To use the latest features for packaging and distributing customizations, use SuiteCloud Development (SDF) instead of SuiteBundler.

SuiteCloud Development Framework lets you create SuiteApps from an integrated development environment (IDE) on your local computer. For more information, see .

To bundle a custom plug-in:

1. Go to Customization > SuiteBundler > Create Bundle.
2. Enter a name for your SuiteApp and any other optional details.
3. In the second step of the Bundle Builder, you don't need to use the **Documentation** field to reference documentation for the custom plug-in type.

   The documentation for your custom plug-in is already attached to the Custom Plug-in Type record, shown in Adding the Default Implementation to NetSuite. Your documentation automatically gets bundled with the Custom Plug-in Type record.
4. In the next step of the Bundle Builder, click the **Plug-ins** folder.
5. Next, click the **Custom Plug-in Types** folder.
6. Under Choose Objects, select the custom plug-in you want to include in your SuiteApp. The custom plug-in includes the default implementation.

   If you (as a solution provider) have developed alternate implementations of your custom plug-in, and you want to include these in the SuiteApp, select the **Plug-ins** > **Custom Plug-in Type Implementations** folder in the Bundler Builder. All your implementations appear in the Choose Objects column.
7. After you've bundled all other SuiteApp objects, click **Next** at the bottom of the screen.
8. In the Set Preferences step of the Bundler Builder, click **Lock on Install** for all objects associated with your custom plug-in type.

Custom Plug-in Implementation

Important: To create your own custom plug-ins and implementation scripts, you need some JavaScript experience and an understanding of SuiteScript. You should write new custom plug-in scripts in SuiteScript 2.0 or SuiteScript 2.1. To get started with SuiteScript, see the help topic SuiteScript 2.1.

Custom plug-in implementations are bundled as part of a SuiteApp. For installation instructions, see the applicable SuiteApp documentation. For more information about creating SuiteApps, see the help topic SuiteBundler Overview.

If you're a solution implementer, review the following sections in order.

- Creating a Custom Plug-in Alternate Implementation
- Adding an Alternate Implementation to NetSuite

Creating a Custom Plug-in Alternate Implementation

Review the custom plug-in type's default implementation and any documentation from the solution provider.

To download a copy of the default implementation:

1. Go to Customization > Plug-ins > Custom Plug-in Type.
2. Click **View** next to the custom plug-in type you want to work with.
3. On the **Methods** subtab, notice the names of the methods exposed in the default implementation. In your alternate implementation, you'll need to use the same method names.
4. On the **Scripts** subtab, click **download** next to the default implementation file. Review this file to understand the default logic for each method.

5. Click **download** next to the documentation file if the solution provider included one. The documentation may provide additional information that explains what the methods are used for and what they return.

You can create a JavaScript file and re-implement the functions from the default implementation by creating an alternate implementation. The alternate implementation overrides the custom plug-in script's default logic. You can't change the function signatures or return types defined in the default implementation. You can only override the logic within the function bodies. Each function that is defined in the default implementation must be fully defined in the alternative implementation.

Tip: Programming Note: Log messages written by using N/log Module functions (log.debug, log.error, log.audit, log.emergency) in a plug-in implementation are tracked on the script record that invokes the plug-in. The log level depends on that script's deployment. You can see log messages by viewing the execution log of the script that invoked the plug-in. For more information about script deployment, see the help topic SuiteScript 2.x Entry Point Script Creation and Deployment.

SuiteScript 2.x Custom Plug-in Alternate Implementation Example

The following is a SuiteScript 2.x example of an **alternate implementation** to the SuiteScript 2.x default implementation example shown in Creating the Default Implementation for a Custom Plug-in:

```
/**
 * @NApiVersion 2.x
 * @NScriptType plugintypeimpl
 */
define(function() {
    return {
        doTheMagic: function(inputObj) {
            return 1234;
        },
        otherMethod: function() {
            return 'sample';
        }
    }
});
```

This sample default implementation fully defines the `doTheMagic` function. When called it in a script, this function always returns a result of 1234. This is different from the default implementation, which accepts two numbers as parameters, adds them together, and returns the result.

Adding an Alternate Implementation to NetSuite

Alternate implementations are stored in the New Plug-in Implementation record type, which is a child record to the Custom Plug-in Type record. Before performing the steps in this procedure, see Creating a Custom Plug-in Alternate Implementation

To create a New Plug-in Implementation record:

1. Go to Customization > Plug-ins > Plug-in Implementations > New.
2. Select the script file that contains your alternate implementation, and then click **Create Plug-in Implementation**.
3. On the New Plug-In Implementation record, enter the following:

- **Name**: Enter a user-friendly name for your implementation. This name appears on the Manage Plug-In Implementations page where administrators activate/deactivate the implementations. For more information, see Managing Custom Plug-in Implementations.
- **ID**: Enter an internal ID for the implementation. If you do not enter an ID, NetSuite creates one for you when you click **Save**.
- **Status**: Set this to either **Testing** or **Release**. Set it to **Released** when you're ready for the implementation to go live in production.
- **Log Level**: Set to the logging level you want for the script.
- **Description**: If you want, add a short description of what the alternate implementation includes.
- **Owner**: This field defaults to the name of the user who's logged in.
- **Inactive**: Specify whether you want the custom plug-in type to be active or inactive.
- On the **Scripts** subtab, enter the following:
  - **Implementation (SuiteScript 1.0 scripts only)**: Select the script file that has your alternate implementation of the custom plug-in type.
  - **Libraries (SuiteScript 1.0 scripts only)**: If you have a library file with utility functions for your alternate implementation, add it here.
- On the **Unhandled Errors** subtab, specify who to notify if script errors occur. By default, the **Notify Script Owner** box is checked.

4. Click **Save**.

You can access all your implementations by going to Customization > Plug-ins > Manage Plug-ins.

Note: You can use SuiteCloud Development Framework (SDF) to manage alternate custom plug-in implementations as part of file-based customization projects. For information about SDF, see .

You can use Copy to Account to copy an individual alternate custom plug-in implementation to another account. When you edit an alternate custom plug-in implementation, you'll see a **Copy to Account** option in the upper right corner. For information about Copy to Account, see the help topic Copy to Account.

Managing Custom Plug-in Implementations

As a NetSuite administrator, you may install a SuiteApp that includes a custom plug-in. If you do nothing, the default implementation logic runs automatically in the SuiteApp.

However, if the developers in your company create alternate implementations of the custom plug-in, use the Manage Plug-Ins page to enable one or more of them. To access this page, go to Customization > Plug-ins > Manage Plug-ins.

Important: You must have Server SuiteScript enabled in your account to access the Manage Plug-In Implementations page.

For custom plug-in types that let you enable and deploy multiple implementations, you'll see a box next to each alternate implementation.

For custom plug-in types that let you deploy only one implementation, you'll see a dropdown list. Select one implementation to enable and deploy, even if there are several implementations to choose from.

After you enable your implementations, click Save.

Important: The solution provider that created the custom plug-in type decides whether single or multiple deployments are supported. As the administrator, you select which implementation or deployment you want enabled in your NetSuite account.

Core Plug-in Overview

A plug-in is functionality, defined by an interface, that can be customized. After the plug-in is installed, a third party can override its default logic with logic that suits their specific needs. The third party does this by defining alternate implementations of the interface

Important: The object-oriented interface is central to the plug-in model. To be more exact, a plug-in **is** an interface. Plug-ins aren't APIs. They don't expose a class's functions or objects. They merely let a third party override the logic defined in the default implementation.

NetSuite develops core plug-ins and usually releases them to partners and customers part of a major release. A core plug-in's interface defines functions that run in the core NetSuite code.

Note: Only core NetSuite developers and call functions defined in a core plug-in's interface, and only within the core NetSuite code.

NetSuite releases each core plug-in with a default implementation. The logic in the default implementation may or may not be available to end users. If applicable, NetSuite may also release a core plug-in with one or more alternate implementations.

After a plug-in is installed in an account, the solution implementer can define one or more alternate implementations. These let the solution implementer customize the core plug-in's logic to suit specific needs. To help with this, NetSuite provides an interface definition that describes the name, parameters, and return type of each function in the core plug-in's interface.

Note: Only plug-in owners can edit alternate implementations. When NetSuite releases a core plug-in with alternate implementations, third parties can't edit them.

When a plug-in's implementations are ready, the NetSuite administrator activates the implementations available for each account. Whether you can one or multiple plug-in implementations at one time depends on how the plug-in is set up. When an implementation is active, function calls made in the core NetSuite code run that implementation's logic.

Core Plug-in Creation

NetSuite develops core plug-ins and releases them to partners or customers usually as part of a major release. A core plug-in's interface defines functions that run in the core NetSuite code. See the following topics:

- Available Core Plug-ins
- Creating a New Plug-in Implementation
- Debugging a Core Plug-in Implementation

A plug-in is functionality defined through an interface. After the plug-in is installed, a third party can replace the plug-in's default logic with logic that suits their needs. For more information about managing plug-ins, see the help topic Managing Plug-ins

Oracle NetSuite develops and releases core plug-ins. The logic in a core plug-in runs in the core NetSuite code. See Core Plug-in Overview for additional information.

Each core plug-in has is own documentation. Refer to the core plug-in's help for details on installing, using, and customizing it The Available Core Plug-ins table links to the help for each generally available core plug-in.

Available Core Plug-ins

The following table lists the available core plug-ins, their descriptions, and the versions of SuiteScript that they support (SuiteScript 1.0, SuiteScript 2.0, or SuiteScript 2.1). For more information about SuiteScript versions, see the help topic SuiteScript Versioning Guidelines.

Name	Description	Supported SuiteScript Version
Custom GL Lines Plug-in	This plug-in adds custom general ledger postings on scriptable transactions.	SuiteScript 1.0
Email Capture Plug-in	This plug-in sets up automated inbound email processing to trigger SuiteScript.	SuiteScript 1.0
Financial Institution Connectivity Plug-in	This plug-in connects directly to financial institutions and automatically receives inbound transmissions so you can automate cash balance reporting and schedule account statement downloads.	SuiteScript 2.0
Bank Connectivity Plug-in	This plug-in makes secure electronic connections to financial institutions for inbound transmissions.	SuiteScript 2.0
Financial Institution Parser Plug-in	This plug-in parses and retrieves bank and credit card data for matching and reconciliation, as well as corporate card charges for expense reporting.	SuiteScript 2.0
Bank Statement Parser Plug-in	This plug-in lets you create and upload your own parsers for different bank and credit card statement formats and store them in NetSuite.	SuiteScript 2.0

Name	Description	Supported SuiteScript Version
	Warning: The Bank Statement Parser Plug-in interface is not supported as of NetSuite 2020.2. It still works, but it's no longer supported. To parse and retrieve more data types, including corporate card expense data for expense reporting, use the Financial Institution Parser Plug-in interface. For details, see the help topic Financial Institution Parser Plug-in Interface Overview.	
Dataset Builder Plug-in	This plug-in lets you create custom datasets for SuiteAnalytics Workbook and is part of the Workbook API.	SuiteScript 2.0 and SuiteScript 2.1
Workbook Builder Plug-in	This plug-in lets you create custom workbooks for SuiteAnalytics Workbook and is part of the Workbook API.	SuiteScript 2.0 and SuiteScript 2.1

Creating a New Plug-in Implementation

The following process is a generic procedure for creating a new plug-in implementation in NetSuite. For details on how to create a new implementation for your specific core plug-in, see the applicable help topic under Available Core Plug-ins.

To create a plug-in implementation, create the script file, then upload it along with any utility files needed.

To create a new plug-in implementation:

1. Review the interface description for your core plug-in.
2. Create a JavaScript file for your implementation and use it to define the logic for the methods in the interface description.
3. In the UI, go to Customization > Plug-ins > Plug-in Implementations > New.
4. In the **Script File** field, open your script file or add a new one.
5. Click **Create Plug-in Implementation**.
6. On the Select Plug-in Type page, click the link for the plug-in type you want to implement.
7. On the New Plug-in Implementation page, enter the following information.

Option	Description
Name	User-friendly name for the implementation. The plug-in implementation appears in the following locations: ■ Manage Plug-ins page – used by administrators to enable and disable the plug-in implementation in their account. ■ Bundle Builder – where you distribute the plug-in implementation to other accounts.
ID	Internal ID for the implementation to use in scripting. If you don't enter an ID, NetSuite creates one for you when you click **Save**.
Status	Current status for the implementation. Choose **Testing** to make it available to the owner, or **Released** to make it accessible to all accounts in production.
Log Level	Select the logging level you want for the script: **Debug**, **Audit**, **Error**, or **Emergency**. These messages appear on the **Execution Log** subtab for the plug-in implementation. Go to Customization > Plug-ins > Plug-in Implementations, select your implementation, and then click the **Execution Log** subtab.

Option	Description
Execute As Role	Select the role the script runs as. The **Execute As Role** field lets you set permissions and restrictions at the role level for the executing script. If you select **Current Role**, the script runs with the permissions of the currently logged-in NetSuite user. **Note:** You can create a custom role during testing to test the plug-in implementation with the correct permissions. The role needs the SuiteScript permission. You can bundle the custom role to distribute it with the plug-in implementation. See the help topics Test the Plug-in Implementation and Bundle the Plug-in Implementation.
Description	Optional description of the implementation. The description appears on the Plug-In Implementations page.
Owner	User account that owns the implementation. By default, it's the name of the logged-in user.
Inactive	Indicates the plug-in implementation doesn't run in the account. You can inactivate a plug-in implementation to temporarily disable it for testing.

Important: You cannot run an implementation as Administrator. You must choose from one of the custom roles listed in the **Execute As Role** field. System Administrator is considered a custom role.

8. On the **Scripts** subtab, in the **Implementation** list, change the JavaScript file for the plug-in implementation, if required.
9. On the **Scripts** subtab, in the **Library Script File** list, select any utility script files or supporting library files that your plug-in script file needs.
10. On the **Unhandled Errors** subtab, specify who to notify if script errors occur.
    - To notify the user who's logged in and running the script, check the **Notify Current User** box.
    - To notify the script owner, check the **Notify Script Owner** box. The **Notify Script Owner** box is checked by default.
    - To notify all administrators, check the **Notify All Admins** box.
    - To notify a group about the error, select the group to notify. Only groups set up in Lists > Relationships > Groups are available.
    - Enter individual email addresses in the **Notify Emails** field, separated by a semi-colon.
11. Click **Save**.
12. Click **Configure** (if available) and enter any needed configuration settings. For example, see the help topic Configuring the Custom GL Lines Plug-in Implementation. Click **Save**.
13. Go to Customization > Plug-ins > Manage Plug-ins.
14. To enable the plug-in implementation, check the box and click **Save**.
15. Test the plug-in implementation.

You can see the list of implementations by going to Customization > Plug-ins > Plug-in Implementations.

Debugging a Core Plug-in Implementation

You can use the SuiteScript Debugger to test your core plug-in implementations.

Core plug-in implementations are debugged in the same way as deployed scripts. To test your implementation in the Debugger, first create a Plug-in Implementation record and set the status to

Testing. For more information, see Creating a New Plug-in Implementation and your specific core plug-in's help.

When you are ready to test your implementation, see the help topics Debugging Deployed SuiteScript 1.0 and SuiteScript 2.0 Server Scripts and Debugging Deployed SuiteScript 2.1 Server Scripts for instructions.